

PYTHON • DATA SCIENCE • EXAM NOTES

NumPy Complete Notes & Guide

Array Properties, Indexing, Reshaping, Broadcasting
Vectorisation & Missing Values

— সহজ ভাষায়, উদাহরণ সহ, পরীক্ষার জন্য প্রস্তুত —

01 · Array Properties

02 · Indexing & Slicing

03 · Reshaping

04 · Modification

05 · Broadcasting

06 · Missing Values

Prepared as a complete revision booklet — theory + code + real-life examples + exam-style notes for every topic.

Table of Contents

বিষয়সূচি

1

Array Properties & Operations

অ্যারের বৈশিষ্ট্য ও অপারেশন

2

Indexing & Slicing Arrays

অ্যারে থেকে ডেটা বাছাই করা

3

Reshaping & Manipulation

অ্যারের shape পরিবর্তন

4

Array Modification

অ্যারে যোগ, বাদ, পরিবর্তন করা

5

Broadcasting & Vectorisation

Automatic shape matching ও দ্রুত calculation

6

Handling Missing Values (NaN)

খালি / অজানা মান সামলানো

Array Properties & Operations

NumPy array কি, কিভাবে বানাবে, আর তার properties গুলো কি কি — সব একসাথে

NumPy Array কি? (কেন লাগে?)

NumPy (Numerical Python) হলো Python-এর সবচেয়ে গুরুত্বপূর্ণ scientific computing library। এর মূল জিনিস হলো **ndarray** (n-dimensional array) — যেটা Python-এর normal list থেকে অনেক দ্রুত এবং memory-efficient। Data Science, Machine Learning, Image Processing — সব জায়গায় NumPy array ব্যবহার হয়।

TIP • কেন List না ব্যবহার করে NumPy Array ব্যবহার করি?

NumPy array গুলো C ভাষায় implement করা, তাই এতে vectorized operation (loop ছাড়া বড় calculation) অনেক গুণ দ্রুত হয়, এবং কম memory লাগে।

Array তৈরি করা

```
import numpy as np

a = np.array([1, 2, 3])           # 1D array
b = np.array([[1,2,3],[4,5,6]])  # 2D array
c = np.zeros((2,3))              # সব 0 দিয়ে ভরা
d = np.ones((3,3))               # সব 1 দিয়ে ভরা
e = np.arange(0,10,2)            # 0 থেকে 10, ধাপে 2
f = np.linspace(0,1,5)           # 0 থেকে 1 এর মধ্যে 5টা সমান দূরত্বের সংখ্যা
g = np.eye(3)                    # Identity matrix
h = np.random.rand(2,2)          # Random values (0-1)
```

Array Properties (গুরুত্বপূর্ণ Attributes)

Property	কাজ কি	Example (a = 2×3 array)
<code>a.shape</code>	প্রতিটা dimension এ কতগুলো element আছে	(2, 3)
<code>a.ndim</code>	Array এর dimension সংখ্যা	2
<code>a.size</code>	মোট কতগুলো element আছে	6
<code>a.dtype</code>	Element গুলোর data type	int64
<code>a.itemsize</code>	প্রতি element কত byte নেয়	8
<code>a.nbytes</code>	পুরো array মোট কত byte নেয়	48
<code>a.T</code>	Transpose (row ↔ column)	shape (3,2)

```
a = np.array([[1,2,3],[4,5,6]])

print(a.shape)    # (2, 3)
```

```
print(a.ndim)      # 2
print(a.size)      # 6
print(a.dtype)     # int64
```

সাধারণ Array Operations

```
a = np.array([1,2,3])
b = np.array([4,5,6])
```

```
a + b  # [5 7 9]
a - b  # [-3 -3 -3]
a * b  # [4 10 18]
a / b  # [0.25 0.4 0.5]
a ** 2 # [1 4 9]
```

```
a.sum()      # মোট যোগফল
a.mean()     # গড়
a.max()      # সবচেয়ে বড়
a.min()      # সবচেয়ে ছোট
a.std()      # Standard deviation
np.sqrt(a)   # বর্গমূল
```

EXAM FOCUS • যা প্রায়ই আসে

- shape vs size এর পার্থক্য বলতে বলে — shape হলো tuple (dimension অনুযায়ী), size হলো মোট element সংখ্যা।
- List আর NumPy array এর মধ্যে পার্থক্য (speed, memory, operations)।
- Array এর dtype কীভাবে বের করে এবং কেন গুরুত্বপূর্ণ (memory efficient calculation)।

REAL-LIFE EXAMPLE

ধরো একটা **ক্লাসের রেজাল্ট শীট** — ৫ জন student, ৩টা subject। এটাকে numpy দিয়ে (5,3) shape এর array বানানো যায়। `marks.mean()` দিয়ে পুরো ক্লাসের average বের করা যায়, `marks.max()` দিয়ে সর্বোচ্চ marks বের করা যায় — এক লাইনেই!

Indexing & Slicing Arrays

Array থেকে নির্দিষ্ট element বা অংশ বের করে আনা

Indexing মানে array এর **একটা নির্দিষ্ট element** বের করা, আর Slicing মানে array এর **একটা অংশ (range)** বের করা। Python list-এর মতোই কাজ করে, কিন্তু NumPy তে multi-dimension এ আরও শক্তিশালী।

1D Array Indexing

```
a = np.array([10, 20, 30, 40, 50])

print(a[0])      # 10   (প্রথম element)
print(a[-1])     # 50   (শেষ element)
print(a[1:4])    # [20 30 40] (index 1 থেকে 3)
print(a[:3])     # [10 20 30] (শুরু থেকে 3টা)
print(a[::2])    # [10 30 50] (এক ঘর পর পর)
print(a[::-1])   # [50 40 30 20 10] (উল্টো করে)
```

2D Array Indexing (Row, Column)

```
m = np.array([[1, 2, 3],
               [4, 5, 6],
               [7, 8, 9]])

print(m[1, 2])    # 6    -> row=1, col=2
print(m[0])       # [1 2 3] -> পুরো প্রথম row
print(m[:, 1])    # [2 5 8] -> পুরো ২য় column
print(m[0:2, 1:3]) # [[2 3], [5 6]] -> sub-matrix
```

QUICK TRICK • মনে রাখার সহজ উপায়

`m[row, column]` — কমা'র আগে row বলে দাও, পরে column। মানে "সব"। যেমন `m[:, 0]` = সব row, শুধু column 0।

Boolean / Conditional Indexing

```
a = np.array([5, 12, 8, 20, 3])

print(a[a > 10])    # [12 20] -> শর্ত মিলা element গুলো
a[a > 10] = 0        # শর্ত মিলা জায়গায় 0 বসিয়ে দিলো
print(a)             # [5 0 8 0 3]
```

Fancy Indexing (একাধিক index একসাথে)

```
a = np.array([10, 20, 30, 40, 50])
idx = [0, 2, 4]
print(a[idx])      # [10 30 50]
```

REAL-LIFE EXAMPLE

একটা **attendance sheet** এ ৩০ জন student এর মধ্যে যাদের marks 40 এর নিচে (fail), তাদের বের করতে:
`students[marks < 40]` — এক লাইনেই পুরো ফেল লিস্ট বের হয়ে যাবে, কোনো loop ছাড়াই!

EXAM TIPS

- Slicing এ শেষ index টা **include হয় না** — `a[1:4]` মানে index 1,2,3 (4 বাদ)।
- Slicing দিয়ে বানানো array আসল array এর সাথে **memory share করে** (view, copy না) — একটাতে পরিবর্তন করলে আরেকটাও বদলে যায়!
- আসলেই আলাদা copy চাইলে `a[1:4].copy()` ব্যবহার করতে হয়।

Reshaping & Manipulation

Array এর shape বা structure পরিবর্তন করা (data একই থাকে)

Reshape মানে array এর element গুলো একই থাকবে, শুধু তাদের **সাজানোর ধরণ (shape)** পরিবর্তন হবে। যেমন একটা 1D array কে 2D বা 3D বানানো যায়, যদি element সংখ্যা মিলে যায়।

reshape() ব্যবহার

```
a = np.arange(1, 13)           # [1 2 3 ... 12]  shape (12,)

b = a.reshape(3, 4)           # 3 row, 4 column
print(b)
# [[ 1  2  3  4]
#   [ 5  6  7  8]
#   [ 9 10 11 12]]

c = a.reshape(2, 3, 2)        # 3D shape: (2,3,2)
d = a.reshape(3, -1)          # -1 দিলে NumPy নিজেই বাকিটা হিসাব করে নেয়
```

WARNING • সাবধান

Reshape করার আগে-পরে **মোট element সংখ্যা same** থাকতে হবে। যেমন 12টা element কে (3,5) shape এ reshape করা যাবে না, কারণ $3 \times 5 = 15 \neq 12$ Error আসবে।

অন্যান্য গুরুত্বপূর্ণ Manipulation Function

Function	কাজ
<code>a.flatten()</code>	Multi-D array কে 1D বানায় (নতুন copy)
<code>a.ravel()</code>	1D বানায় (কিন্তু original এর view, memory share করে)
<code>a.T</code> / <code>np.transpose(a)</code>	Row-Column অদল বদল
<code>np.concatenate([a,b])</code>	দুইটা array জোড়া লাগানো
<code>np.vstack([a,b])</code>	Vertically (উপর-নিচ) জোড়া লাগানো
<code>np.hstack([a,b])</code>	Horizontally (পাশাপাশি) জোড়া লাগানো
<code>np.split(a, 3)</code>	Array কে ৩ ভাগে ভাগ করা
<code>np.expand_dims(a, axis=0)</code>	নতুন একটা dimension যোগ করা
<code>np.squeeze(a)</code>	Size-1 dimension গুলো বাদ দেওয়া

```
a = np.array([[1,2],[3,4]])
b = np.array([[5,6],[7,8]])
```

```
print(np.vstack([a,b]))  
# [[1 2]  
#   [3 4]  
#   [5 6]  
#   [7 8]]  
  
print(np.hstack([a,b]))  
# [[1 2 5 6]  
#   [3 4 7 8]]
```

REAL-LIFE EXAMPLE

একটা **28×28 pixel** এর ছবি (grayscale) মূলত 2D array। কিন্তু Machine Learning model এ ইনপুট দিতে হলে সেটাকে 1D বানাতে হয় — `image.reshape(784)` বা `image.flatten()` ব্যবহার করে। এভাবেই Deep Learning এ image data প্রস্তুত করা হয়।

EXAM TIPS

- `flatten()` সবসময় নতুন copy বানায়, `ravel()` যতটা সম্ভব original memory ব্যবহার করে (faster, কিন্তু risky)।
- Reshape এ -1 এর মানে কি সেটা exam এ প্রায়ই জিজ্ঞেস করে — মানে "এই dimension NumPy নিজে হিসাব করবে"।

Array Modification

Array এ element যোগ, বাদ, আপডেট বা সাজানো

Array Modification মানে হলো — একটা array এর ভেতরের data পরিবর্তন করা: নতুন element যোগ করা, পুরনো element বাদ দেওয়া, sort করা, বা কোনো value বদলে দেওয়া।

Value পরিবর্তন করা (Update)

```
a = np.array([1, 2, 3, 4, 5])

a[0] = 100          # নির্দিষ্ট index পরিবর্তন
a[1:3] = [20, 30]   # একসাথে একাধিক পরিবর্তন
print(a)             # [100 20 30 4 5]
```

Element যোগ / বাদ দেওয়া

```
a = np.array([1, 2, 3])

b = np.append(a, [4, 5])   # শেষে যোগ -> [1 2 3 4 5]
c = np.insert(a, 1, 99)    # index 1 এ ঢোকানো -> [1 99 2 3]
d = np.delete(a, 0)        # index 0 বাদ -> [2 3]
```

WARNING • গুরুত্বপূর্ণ

NumPy array এর size **fixed** থাকে — Python list এর মতো সরাসরি বড়/ছোট করা যায় না। `append()`, `insert()`, `delete()` — এগুলো আসলে **নতুন একটা array বানিয়ে return করে**, original array কে সরাসরি পরিবর্তন করে না।

Sorting করা

```
a = np.array([5, 2, 8, 1, 9])

print(np.sort(a))      # [1 2 5 8 9] ছোট থেকে বড়
a.sort()               # নিজের ভেতরেই sort করে ফেলবে
print(np.sort(a[::-1])) # বড় থেকে ছোট
```

Copy vs View — খুব Important!

Copy	View
Original থেকে সম্পূর্ণ আলাদা, নতুন memory	Original এর সাথে memory share করে
একটাতে পরিবর্তন করলে অন্যটা বদলায় না	একটা বদলালে আরেকটাও বদলে যায়
<code>b = a.copy()</code>	<code>b = a.view()</code> বা <code>b = a[1:3]</code> (slicing)

```
a = np.array([1,2,3])
b = a          # same object (reference)!
c = a.copy()   # সম্পূর্ণ আলাদা copy

b[0] = 999
print(a)      # [999 2 3]   -> a ও বদলে গেছে!
print(c)      # [1 2 3]    -> c অপরিবর্তিত
```

REAL-LIFE EXAMPLE

একটা **To-Do list app**-এ নতুন task add করা মানে `np.append()`, কোনো task complete হলে delete করা মানে `np.delete()`। আর priority অনুযায়ী task সাজাতে চাইলে `np.sort()` ব্যবহার হয়।

EXAM TIPS

- Copy আর View এর পার্থক্য প্রায় প্রতি exam এ আসে — মনে রাখো: **Slicing = View, .copy() = Copy**।
- NumPy array এর **fixed size** — এই কথাটা লিখতে ভুলো না।

Broadcasting & Vectorisation

ভিন্ন shape এর array একসাথে কাজ করানো, আর loop ছাড়া দ্রুত calculation

Broadcasting কি?

Broadcasting হলো NumPy এর একটা ক্ষমতা, যেটা দিয়ে **ভিন্ন shape এর array গুলোকে** একসাথে operation (add, subtract, multiply) করা যায়। ছোট array কে NumPy স্বয়ংক্রিয়ভাবে বড় array এর shape এ "কল্পনায়" stretch করে নেয় — বাস্তবে কোনো memory copy হয় না।

```
a = np.array([1, 2, 3])
b = 5
print(a + b)      # [6 7 8]  -> b সবগুলোর সাথে যোগ হলো

m = np.array([[1,2,3],[4,5,6],[7,8,9]]) # shape (3,3)
r = np.array([10,20,30])                # shape (3,)
print(m + r)
# [[11 22 33]
#  [14 25 36]
#  [17 28 39]]  -> প্রতি row তে r যোগ হয়ে গেলো
```

Broadcasting Rule

দুইটা array এর shape ডানদিক থেকে মিলাতে হয়। প্রতিটা dimension এর জন্য নিচের যেকোনো একটা সত্যি হতে হবে:

✓ Dimension দুইটা সমান

✓ একটা dimension = 1

✓ একটা dimension না থাকা (missing)

```
# Compatible ✓
(3,3) & (3,)    -> ✓
(3,3) & (1,3)   -> ✓
(3,3) & (3,1)   -> ✓

# Not Compatible ✗
(2,3) & (2,)    -> ✗ ValueError
```

Vectorisation কি?

Vectorisation মানে হলো — for-loop না লিখে, সরাসরি পুরো array-তে একবারে operation করা। এতে code অনেক ছোট এবং দ্রুত হয় (কারণ NumPy ভেতরে C ভাষায় optimized loop চালায়)।

SLOW WAY · Loop দিয়ে

```
result = []
for x in prices:
    result.append(x - x*0.1)
```

FAST WAY · Vectorised

```
result = prices - prices*0.1
```

REAL-LIFE EXAMPLE

একটা দোকানে ৫টা product-এ একসাথে ১০% ছাড় দিতে: $\text{final} = \text{prices} - \text{prices} * 0.10$ । Machine Learning এ data normalization করতে: $(\text{data} - \text{mean}) / \text{std}$ — এখানেও broadcasting কাজ করছে, কারণ mean, std এর shape data থেকে ছোট।

EXAM TIPS

- Broadcasting এর সময় memory তে **copy তৈরি হয় না** — এটা memory efficient।
- Vectorisation এর মূল benefit: **speed** (loop এর দরকার নেই) এবং **ছোট, পরিষ্কার code**।
- Broadcasting rule অনুযায়ী কোন দুইটা shape compatible সেটা বের করতে বলতে পারে — ডানদিক থেকে মিলাও, মনে রাখো।

Handling Missing Values (NaN)

Array এ খালি বা অজানা মান (NaN) কীভাবে খুঁজে বের করে সামলাবে

NaN কি?

NaN মানে **Not a Number** — এটা দিয়ে বোঝানো হয় কোনো data missing বা undefined। Real-life dataset এ প্রায়ই কিছু value খালি থাকে (যেমন কোনো student এর একটা subject এর marks নাই) — সেটাকে numpy তে `np.nan` দিয়ে represent করা হয়।

```
a = np.array([1, 2, np.nan, 4, np.nan])
print(a)      # [1. 2. nan 4. nan]
```

WARNING · Important Fact

`np.nan` সবসময় **float type**। তাই যে array তে NaN থাকবে সেটা automatic float dtype হয়ে যাবে, int থাকে না। এছাড়া `np.nan == np.nan` সবসময় **False** হয় — সমতা দিয়ে চেক করা যায় না!

NaN খুঁজে বের করা

```
a = np.array([1, 2, np.nan, 4, np.nan])

print(np.isnan(a))      # [False False True False True]
print(np.isnan(a).sum()) # 2  -> মোট কতগুলো NaN আছে
print(a[np.isnan(a)])   # [nan nan]  -> শুধু NaN গুলো
print(a[~np.isnan(a)])  # [1. 2. 4.]  -> NaN বাদে সব (~ মানে "not")
```

Missing Value সামলানোর উপায়

উপায়	Code	কখন ব্যবহার হয়
বাদ দেওয়া (Remove)	<code>a[~np.isnan(a)]</code>	Missing value কম হলে
নির্দিষ্ট মান বসানো	<code>np.nan_to_num(a, nan=0)</code>	0 বা কোনো default value দিতে চাইলে
গড় (Mean) দিয়ে ভরা	<code>np.nanmean(a)</code>	Statistical imputation
NaN বাদে হিসাব করা	<code>np.nansum()</code> , <code>np.nanmax()</code>	Calculation এ NaN কে ignore করতে

```
a = np.array([1, 2, np.nan, 4, np.nan])

print(np.nanmean(a))    # 2.333...  -> NaN বাদ দিয়ে গড়
print(np.nansum(a))     # 7.0

b = np.where(np.isnan(a), np.nanmean(a), a)
print(b)                # [1. 2. 2.33 4. 2.33]  -> NaN জায়গায় mean বসে গেলো
```

TIP • নরমাল **Function** গুলো **NaN** থাকলে সমস্যা করে

`a.mean()` যদি array তে NaN থাকে, তাহলে result ও nan হয়ে যায়! তাই সবসময় `np.nanmean()`, `np.nansum()` এর মতো "nan-safe" function ব্যবহার করা উচিত।

REAL-LIFE EXAMPLE

একটা **survey dataset** এ কিছু মানুষ তাদের বয়স লেখেনি (missing)। সেই column এর গড় বের করতে `np.nanmean(ages)` ব্যবহার করলে missing value গুলো বাদ দিয়েই সঠিক গড় বের হবে। Machine Learning model train করার আগে এই missing value সামলানো (Data Cleaning / Imputation) একটা বাধ্যতামূলক ধাপ।

EXAM TIPS

- `np.nan` এর data type সবসময় **float** — মনে রাখা জরুরি।
- `np.isnan()` দিয়ে check করে, সরাসরি `== np.nan` দিয়ে কখনো check করা যায় না।
- `mean()` আর `nanmean()` এর পার্থক্য বলতে পারা জরুরি (NaN থাকলে normal function ফলাফল NaN দেয়)।

Quick Revision Summary

সব টপিক এক নজরে — শেষ মুহূর্তের রিভিশনের জন্য

01 • Array Properties

- ndarray = NumPy এর মূল object
- shape, ndim, size, dtype — গুরুত্বপূর্ণ attribute
- List থেকে দ্রুত ও memory efficient

02 • Indexing & Slicing

- a[row, col] দিয়ে নির্দিষ্ট element
- Slicing এ শেষ index বাদ যায়
- Boolean indexing: a[a > 10]

03 • Reshaping

- reshape() — shape বদলায়, data একই
- Total element সংখ্যা মিলতে হয়
- flatten() vs ravel() (copy vs view)

04 • Array Modification

- append(), insert(), delete()
- NumPy array এর size fixed
- Copy vs View মনে রাখা জরুরি

05 • Broadcasting

- ভিন্ন shape এ automatic matching
- Rule: right থেকে shape মিলাও
- Vectorisation = loop ছাড়া দ্রুত calculation

06 • Missing Values

- np.nan সবসময় float type
- np.isnan() দিয়ে খুঁজে বের করা
- nanmean(), nansum() — nan-safe function

EXAM FOCUS • সবচেয়ে বেশি যা জিজ্ঞেস করা হয়

1. Copy vs View এর পার্থক্য এবং উদাহরণ।
2. Broadcasting এর rule এবং কোন shape গুলো compatible তা বের করা।
3. reshape() এ -1 এর অর্থ কি এবং কখন error হয়।
4. Boolean indexing দিয়ে condition অনুযায়ী data filter করা।
5. np.nan এর বৈশিষ্ট্য এবং nanmean() / nansum() এর ব্যবহার।